



INTERNSHIP CHALLENGE

Build an app. Get **hired.**

No traditional interviews. Build something small in one week, then walk us through how you built it. The project is yours to keep.

DURATION

3 months

flexible

ALLOWANCE

RM2,000

per month

WHERE

Bangsar

on-site, KL

DEADLINE

16 Aug

11:59pm, 2026

shortcut.my · Paid internship · CS, IT & Software Engineering students

THE OPPORTUNITY

What you're building toward

A paid internship where you work on real software for real clients. Our interns don't fetch coffee. You join a product team and ship code that real users interact with.

Past intern work has included

- Banking and fintech platforms
- E-commerce applications
- Transport and logistics systems
- AI-powered product features

Duration: 3 months (flexible around your university's requirements)

Allowance: RM2,000 / month

Arrangement: On-site, office near LRT Bangsar

For: Students in CS, IT, Software Engineering, or related fields

Start dates: November 2026 to February 2027 (flexible around your university's requirements)

THE PROCESS

How it works

1. Pick a topic, or propose your own
2. Build it within the build window
3. Submit by **16 August 2026, 11:59pm**
4. Our developers may call you during review week to talk through your build
5. Get shortlisted
6. Pitch to the team at our Bangsar office, where offers go out on the day

CHOOSE YOUR BUILD

Topics

Five starting points below. These are harder than they look. Each one hides a real problem, an algorithm, a correctness trap, or a genuine calculation that most apps get wrong. That hidden problem is the whole point. Don't try to build all of it. Nail the core, go deep on the hard part, and be ready to explain it. You're not limited to these. If you have a better idea, or want to put your own spin on one, go for it. We want to see what excites you.

01

Errand Route Planner

Plan a set of errands and get the smartest order to run them in.

CORE FEATURES

Add stops, and an ordered route.

THE HARD PART

Ordering stops to minimise travel is a classic hard problem. Even a rough, sensible solution requires you to think about the trade-offs and explain why your ordering beats just doing the errands in the order they were added.

02

Renewal Reminder

One place for road tax, licence, passport, insurance, and subscription renewals that nudges you before anything lapses.

CORE FEATURES

Add items with their renewal dates, and a reminder view of what's coming up.

THE HARD PART

Everything renews on a different cycle, and a reminder is only useful if it lands with enough lead time to act. Deciding when to nudge, and not drowning the user in alerts, is the actual design problem.

03

Cash-Flow Forecaster

Plug in your income and recurring bills, and see your projected balance across the month so you know when things get tight.

CORE FEATURES

Add income and recurring expenses, and a forecast view.

THE HARD PART

Bills land on different cycles (weekly, monthly, that one annual renewal), and the useful output is the exact day your balance dips lowest. Modelling those overlapping cycles correctly is where it lives.

04

Cheapest Basket

Enter your shopping list and find out which shop, or which split between two shops, gets you the whole basket for the least.

CORE FEATURES

Keep prices for your regular shops, and an optimised basket result.

THE HARD PART

One shop is cheaper on some items and pricier on others, so the best answer isn't obvious. And a second stop

only pays off if the saving beats the extra trip. Deciding that trade-off is the challenge, not the list.

05

Savings Goal Allocator

Set a few savings goals with target amounts and dates, and see how to split your spare cash each month to hit them.

CORE FEATURES

Add goals with targets and deadlines, and a monthly allocation plan.

THE HARD PART

Your surplus is limited and the goals compete. The useful app works out a split that lands each goal on time, and tells you plainly which goals aren't reachable rather than pretending they are.

Or build something else entirely. If you have an idea that excites you, or a problem you keep running into and wish someone would fix, build that instead. A well-chosen idea of your own counts just as much as the five above. Pick the thing you'll do your best work on.

Tech stack is your call. Web, mobile, desktop, or CLI. Use whatever you're comfortable with.

READ THIS CAREFULLY

How to approach this

This matters more than the topic you pick.

Plan for roughly 8 to 12 hours of work. The build window exists so you can fit this around classes and life, not so you can pour 60 hours into something huge. We are not scoring effort. A small, complete, well-understood app beats a large, impressive one you can't explain.

Go deep, not wide. Two solid features you fully understand are worth more than ten half-built ones. Pick one meaningful problem and do it properly.

Understand what you build. By the end, you should be able to explain what your app does, why you made the choices you made, how it works under the hood, and where it falls short. If you can't explain a part of it, that part isn't finished.

Here's the difference we're looking for, made concrete:

Candidate A

Spent 60 hours. The app looks polished and packed with features. But they can't explain how it works or why they made key decisions.

Candidate B

Spent 12 hours. The app is small but the core workflow is complete. They followed a clear process, can explain every choice, and checked that it actually works.

We hire **Candidate B.**

THE BAR

What we're looking for

We care about how you think, not just what you build. Specifically:

- **A systematic approach.** Evidence that you planned, worked in steps, and made deliberate decisions rather than piling things on until it ran.
- **Understanding the problem.** You grasp what the app is for, who uses it, and where its limits are.
- **Understanding your own solution.** You can explain how it works, even in broad strokes, and why it's built the way it is.
- **Sound reasoning.** You can talk through the choices you made, the alternatives you weighed, and the trade-offs.
- **Honest reflection.** You know what's unfinished, what's uncertain, and what you'd check before real users touched it.
- **Ownership.** You stand behind everything you submit, including the parts an AI helped you write.

What we're not looking for

Skip these. They won't help your score, and chasing them usually hurts it:

- **Decorative polish.** A beautiful UI on top of a workflow you don't understand.
- **Feature volume.** More features is not more impressive. It's usually less.
- **Cinematic demos.** A flashy video won't cover for a shaky product. Keep the demo simple and honest.
- **Time-maxxing.** Spending the whole window on this to look committed. We'd rather you spent 10 focused hours.

BE HONEST ABOUT IT

Using AI

Use AI. We expect it, and how you use it is a real part of how we evaluate you. Don't hide it, and don't be shy about it in your pitch. We would much rather hear "I used AI here, and here's how I checked it" than have you pretend you didn't.

What we want to see is *thoughtful* use, matched to the task:

- Delegating the right work to AI, and keeping the parts you should own
- Reading and understanding what it gives you before you ship it
- Verifying that its output actually works

- Catching, rejecting, or reworking suggestions when they're wrong
- Avoiding complexity you don't need

AI use runs on a spectrum, from simple autocomplete, to asking a chatbot questions, to letting an agent run and reviewing the result, to more structured setups. You don't need the fanciest workflow. Someone using simple tools carefully will beat someone running an elaborate agent they can't explain. Reach for the approach that fits the job, and be ready to talk about why.

Show your working

Commit as you go. A commit history that shows how your work progressed helps us understand your process, and makes your pitch easier. This isn't decisive and it's fine if you're new to Git, but small, meaningful commits beat one giant “final” dump at the end.

WHAT TO HAND IN

Submission requirements

1. **Working application.** Hosted online (Vercel, Netlify, or similar) or with clear setup instructions so we can run it.
2. **Source code.** A GitHub/GitLab/Bitbucket repo, public or invite us. Document any complex logic in the repo.
3. **Documentation.** 1 to 2 pages on your thinking, not a manual. See below.
4. **Demo video.** 3 to 5 minutes, voice-over recommended, face optional. Upload to YouTube (unlisted) or Google Drive and share the link. Focus on what you built, how it works, and why you made your key decisions.

YOUR DOCUMENTATION SHOULD COVER

- How you planned and approached the app
- Why you chose your tools, frameworks, or libraries
- Your main technical decisions and the reasoning behind them
- 1 to 2 flowcharts for key feature flows
- A simple architecture overview and the tech stack used
- Where you used AI, and how you checked its output

Submit everything here: <https://forms.gle/SkLx43JSJcmyWVgC6>

IF YOU'RE SHORTLISTED

The pitch session

You'll present to the team at our Bangsar office on **26 August (tentative)**. Please keep the date free for now. It's short, 5 to 10 minutes, and it's a conversation, not an interrogation. Offers are made on the day.

Before anything else: relax about AI. Using it is expected. Sharing how you used it helps you, it doesn't count against you.

To give you a sense of what we'll ask, here are the kinds of questions that come up:

- Walk us through how you'd add or change a feature in your app.
- Why did you choose this approach?
- What assumptions does your solution make?
- How did you check that it behaves correctly?
- What alternatives did you consider?
- Did you reject or rework any AI suggestions? Why?
- What risks or uncertainties are left?
- If you released this tomorrow, what would you check first?
- What did you decide not to build, because it mattered less?
- Where might a user get confused or lose trust in the flow?
- What changed once you actually used the app yourself?

None of these have a single right answer. We're listening for how you reason.

KEY DATES

Timeline

WHAT	WHEN
Brief released	6 August 2026
Open Q&A call	Friday 7 August, 2:00pm
Build period	6 to 16 August 2026
Submission deadline	16 August 2026, 11:59pm
Review week (developer calls possible)	17 to 21 August 2026
Shortlisted presentations	26 August 2026 (tentative), offers on the day

- To join the open call, reply to the email this brief came with to confirm. We'll answer any questions you have about the challenge or interning at Shortcut Asia.
- After you submit, one of our developers may call you during review week to walk through your build before we shortlist. It's a friendly chat, not a test, so no prep needed.

GOOD TO KNOW

FAQ

Do we build something new, or can we submit an old project?

If you go with one of our topics, build it fresh from scratch. Or you're welcome to submit a passion project you've already built.

Who owns the work?

You do, fully. Keep it as a portfolio piece, keep improving it, or turn it into something bigger.

How much time should this take?

Plan for roughly 8 to 12 hours. The build window is for scheduling flexibility, not for maximizing hours. We don't reward time spent.

Do I have to use the suggested topics?

No. Use them, remix them, or propose your own. All submissions are judged the same way.

Is there a required tech stack?

No. TypeScript is recommended, but use what you're comfortable with. Just make sure you can finish in time.

Two features, or more?

Two solid, well-built features are enough. Go deeper, not broader. Add more only if it genuinely strengthens the app and you can still finish.

Can it be a CLI instead of a GUI?

Yes. Web, mobile, desktop, or CLI all work. Just justify the choice: the use case, the target user, and why that interface fits.

Am I allowed to use AI?

Yes, and we expect it. How you use it is part of the evaluation, so use it thoughtfully and be ready to explain what you delegated, what you kept, and how you checked the output.

Should I make my commit history look impressive?

No need to game it. Just commit naturally as you work. Meaningful, incremental commits help us follow your process, but they don't decide the outcome.

What carries the most weight in scoring?

How you think. We review the code, the product, and your presentation, but the deciding factor is whether you understand what you built and can reason about it.

Does design and UX matter, or just functionality?

Both. We build products, so it has to work and the experience has to make sense. But polish never substitutes for understanding.

READY?

**Pick a topic.
Start building.
Show us how you think.**

Submission deadline: **16 August 2026, 11:59pm**

Submit at forms.gle/SkLx43JSJcmyWVgC6

 **SHORTCUT ASIA**